

BEE 4750 Homework 1: Introduction to Using Julia

Name: Matthew Burgos

ID: mb2557

Due Date

Thursday, 9/11/25, 9:00pm

Overview

Instructions

- Problem 1 consist of a series of code snippets for you to interpret and debug. You will be asked to identify relevant error(s) and fix the code.
- Problem 2 asks you to write code to implement several simple functions using more general programming and some which are unique to Julia (which might mean that you need to look at the documentation or use Julia's help function, which is accessed with `?`, e.g. `?sum` for help with the `sum()` function) or to analyze Julia syntax.
- Problem 3 asks you to convert a verbal description of a wastewater treatment system into a Julia function, and then to use that function to explore the impact of different wastewater allocation strategies.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `~/Desktop/bee4750-hw1-mb2557`

Standard Julia practice is to load all needed packages at the top of a file. If you need to load any additional packages in any assignments beyond those which are loaded by default, feel free to add a `using` statement, though [you may need to install the package](#).

```
using Random
using Plots
using GraphRecipes
using LaTeXStrings
using Distributions
```

```
# this sets a random seed, which ensures reproducibility of random number generation. You sh
Random.seed!(1)
```

TaskLocalRNG()

Problems (Total: 30 Points)

Problem 1 (12 points)

The following subproblems all involve code snippets that require debugging. You are encouraged to use online resources (*e.g.* Julia documentation and forums, Stack Overflow, Reddit, etc) to help find diagnose error messages and find solutions, but make sure that you clearly document which resources you used and how you used them.

Problem 1.1

You've been tasked with writing code to identify the minimum value in an array. You cannot use a predefined function. Your colleague suggested the function below, but it does not return the minimum value.

```
function minimum(array)
    # initialize the minimum value counter
    # old code: min_value = 0
    min_value = array[1]
    # update minimum values
    for i in 1:length(array)
        if array[i] < min_value
            min_value = array[i]
```

```

        end
    end
    # return found minimum
    return min_value
end

array_values = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
@show minimum(array_values);

```

```
minimum(array_values) = 78
```

What is the logical flaw in the code? Fix it and show that your fixed code solves the problem.

The logical flaw is that `min_value` is initialized to zero. If none of the values in the array are less than 0, then it will state that the minimum value is 0. Instead, I initialized `min_value` to the first index. This ensure that the `min_value` is always a value that exists in the array. For the example array, `minimum()` returns 78 as expected.

Problem 1.2

Your team is trying to compute the average grade for your class, but the following code produces an `UndefVarError`.

```

# enter student grade vector
student_grades = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
# compute class average
function class_average(grades)
    average_grade = mean(student_grades)
    return average_grade
end

# define global var average_grade
average_grade = class_average(student_grades)
@show average_grade;

```

```
average_grade = 94.4
```

What does this `UndefVarError` mean? Why does this code produce one? Fix the error and solve the problem.

`UndefVarError` means that a variable cannot be accessed. Either the variable has not been defined or the variable is not accessible within the current scope. In this case, there is an error

because the variable `average_grade` has been defined locally (within the function), but it is trying to be accessed globally. I fixed the problem by defining a global variable, `average_grade`, to be equal to `class_averages(student_grades)`. Now, there is a variable equal to the average grade that can be accessed globally. The result is 94.4 as expected.

Source to define `UndefValError`: <https://docs.julialang.org/en/v1/manual/variables-and-scoping/>

Problem 1.3

Your team wants to know the expected payout of an old Italian dice game called *passadieci* (which was analyzed by Galileo as one of the first examples of a rigorous study of probability). The goal of *passadieci* is to get at least an 11 from rolling three fair, six-sided dice. Your strategy is to compute the average wins from 1,000 trials, but the code you’ve written below produces a `MethodError`.

```
# function to simulate a passedieci roll: 3 6-sided dice
function passedieci()
    # this rand() call samples 3 values from the vector [1, 6]
    roll = rand(1:6, 3)
    return roll
end
# set number of trials and initialize outcome vector
n_trials = 1_000
# old code: outcomes = zero(n_trials)
outcomes = zeros(n_trials)
# simulate number of passedieci rolls and count wins
for i = 1:n_trials
    # old code: outcomes[i] = (sum(passadieci()) > 11)
    outcomes[i] = (sum(passadieci()) >= 11)
end
win_prob = sum(outcomes) / n_trials # compute average number of wins
@show win_prob;
```

```
win_prob = 0.502
```

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

`MethodError` means that “a method with the required type signature does not exist in the given generic function” or a method call does not have the proper arguments. This code produces a `MethodError` because `outcomes` is not initialized correctly. `outcomes` is initialized to the integer 0 instead of an array. `zero(x)` returns the additive identity for the type of

x. Therefore, when trying to call `outcomes[i]`, there is an error because integers do not have indices. I fixed the error by changing `zero(n_trials)` to `zeros(n_trials)`, properly initializing `outcomes` to an array of 1000 zeros. Also, the number of wins was only counting sums greater than 11 when it should have been counting sums greater than or equal to 11. I fixed this in the code and now the probability is correctly around 0.5.

Source to define `MethodError`: <https://www.jlhub.com/julia/manual/en/function/MethodError>

Source to define `zero()`: <https://www.jlhub.com/julia/manual/en/function/zero>

Source to define `zeros()`: <https://www.jlhub.com/julia/manual/en/function/zeros>

Source to check rules of `passadieci`: <https://en.wiktionary.org/wiki/passadieci>

Problem 1.4

You're interested in writing some code to remove the mean of a vector from all of its components. You've written the following code and tried to test it on a random vector, but your code returns a `MethodError`.

```
# function to remove mean from a vector
function remove_mean(vect)
    # function to compute the mean
    function compute_mean(vect)
        element_sum = 0 # initialize sum
        # compute mean and return
        for v in vect
            element_sum += v
        end
        return element_sum / length(vect)
    end

    m = compute_mean(vect) # compute mean

    # return demeaned vector
    # old code: return vect - m
    for i in eachindex(vect)
        vect[i] -= m
    end
    return vect
end
```

```
random_vect = rand(1_000)
@show remove_mean(random_vect)
```

```
remove_mean(random_vect) = [0.41919142411294885, -0.04477120169153992, -0.15987063495003362,
```

```
1000-element Vector{Float64}:
```

```
 0.41919142411294885
-0.04477120169153992
-0.15987063495003362
-0.3165390583665356
-0.04345183269168251
 0.4600428661267356
-0.4814808404373695
 0.05265958609672772
-0.019033198532207418
 0.11752715148103854
-0.2530763598508532
-0.3987461186978969
-0.4051668577883766
```

```
 0.4389503908252199
-0.2337089015832472
 0.13250675888374663
 0.1701029484937252
-0.45332724205642416
-0.4743008238822817
-0.4129715382326393
 0.23830378602543056
 0.06271140871181402
 0.11506229654060751
-0.1638906511382653
-0.0897920909169232
```

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

This `MethodError` is saying that Julia doesn't recognize a method with the format: `vector - integer`. In this case, the `-` command should be used between floats and ints. This code produced a `MethodError` because there is no subtraction command between vectors and integers. To fix the error, I used a for-loop to subtract the mean from each element in `vect`. I returned the new `vect`, which contains values that waver around 0. This makes sense because the mean of values in the new vector should be 0.

Problem 2 (18 points)

Cheap Plastic Products, Inc. is operating a plant that produces $100\text{m}^3/\text{day}$ of wastewater that is discharged into Pristine Brook. The wastewater contains $1\text{kg}/\text{m}^3$ of YUK, a toxic substance. The US Environmental Protection Agency has imposed an effluent standard on the plant prohibiting discharge of more than $20\text{kg}/\text{day}$ of YUK into Pristine Brook.

Cheap Plastic Products has analyzed two methods for reducing its discharges of YUK. Method 1 is land disposal, which costs $X_1^2/20$ dollars per day, where X_1 is the amount of wastewater disposed of on the land (m^3/day). With this method, 20% of the YUK applied to the land will eventually drain into the stream (*i.e.*, 80% of the YUK is removed by the soil).

Method 2 is a chemical treatment procedure which costs $\$1.50$ per m^3 of wastewater treated. The chemical treatment has an efficiency of $e = 1 - 0.005X_2$, where X_2 is the quantity of wastewater (m^3/day) treated. For example, if $X_2 = 50\text{m}^3/\text{day}$, then $e = 1 - 0.005(50) = 0.75$, so that 75% of the YUK is removed.

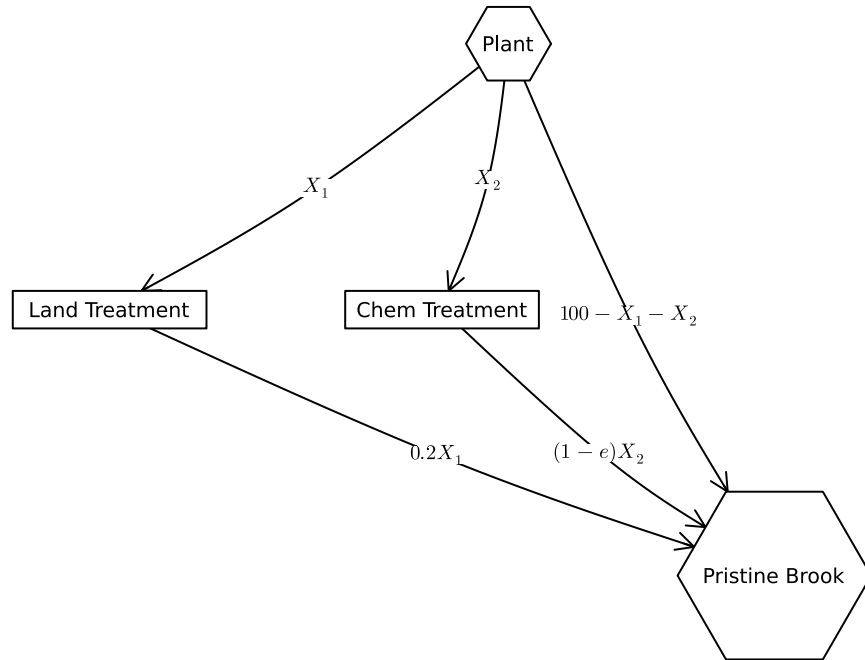
Cheap Plastic Products is wondering how to allocate their wastewater between these three disposal and treatment methods (land disposal, chemical treatment, and direct disposal) to meet the effluent standard while keeping costs manageable.

The flow of wastewater through this treatment system is shown in Figure 1. Modify the edge labels (by editing the `edge_labels` dictionary in the code producing Figure 1) to show how the wastewater allocations result in the final YUK discharge into Pristine Brook. For the `edge_label` dictionary, the tuple (i, j) corresponds to the arrow going from node i to node j . The syntax for any entry is $(i, j) \Rightarrow \text{"label text"}$, and the label text can include mathematical notation if the string is prefaced with an L, as in `L"x_1"` will produce x_1 .

```
A = [0 1 1 1;
      0 0 0 1;
      0 0 0 1;
      0 0 0 0]

names = ["Plant", "Land Treatment", "Chem Treatment", "Pristine Brook"]
# modify this dictionary to add labels
edge_labels = Dict((1, 2) => L"x_1", (1,3) => L"x_2", (1, 4) => L"100-x_1-x_2", (2, 4) => L"0", (3, 4) => L"0")
shapes=[:hexagon, :rect, :rect, :hexagon]
xpos = [0, -1.5, -0.25, 1]
ypos = [1, 0, 0, -1]

p = graphplot(A, names=names,edgelabel=edge_labels, markersize=0.15, markershapes=shapes, margin=0.5)
display(p)
```



I added labels to the edge_labels to show each flow in terms of kg/day of YUK. Because the concentration of YUK is 1 kg/m³, 1 m³/day of wastewater = 1 kg/day of YUK.

Problem 2.1

Formulate a mathematical model for the treatment cost and the amount of YUK that will be discharged into Pristine Brook based on the wastewater allocations. This is best done with some equations and supporting text explaining the derivation. Make sure you include, as additional equations in the model, any needed constraints on relevant values. You can find some basics on writing mathematical equations using the LaTeX typesetting syntax [here](#), and a cheatsheet with LaTeX commands can be found on the course website's [Resources page](#).

$$\text{Treatment cost} = \text{land treatment cost} + \text{chem treatment cost} + \text{direct disposal cost}$$

$$\text{Treatment cost} = X_1^2/20 + 1.5X_2 + 0$$

$$\text{Treatment cost} = X_1^2/20 + 1.5X_2 \text{ (dollars/day)}$$

The total treatment cost is the sum of all three methods of treatment. As specified above, land treatment is $X_1^2/20$ dollars/day while chem treatment is 1.5 dollars/m³ or $1.5X_2$ dollars/day. Direct disposal is assumed to be free because there is no information on cost.

$YUK\ discharge = land\ treatment\ discharge + chem\ treatment\ discharge + direct\ disposal$

$YUK\ discharge = 0.2X_1 + (1 - e)X_2 + (100 - X_1 - X_2)$

$YUK\ discharge = 0.2X_1 + (1 - (1 - 0.005X_2))X_2 + (100 - X_1 - X_2)$

$YUK\ discharge = 0.2X_1 + (0.005X_2)X_2 + (100 - X_1 - X_2)$

$YUK\ discharge = 0.2X_1 + 0.005X_2^2 + (100 - X_1 - X_2)$

where $X_1 + X_2 \leq 100$ and $YUK\ discharge < 20\ kd/day$

The total YUK discharge is the sum of all three discharge sources. As specified above, land treatment discharges 20% of the original YUK ($0.2X_1$). Chem treatment discharges $(1-e)\%$ of the original YUK $((1-e)X_2)$. Discharge of direct disposal is equal to the flow $(100 - X_1 - X_2)$ times the concentration ($1\ kg/m^3$).

Problem 2.2

Implement your systems model as a Julia function which computes the resulting YUK discharge and cost for a particular treatment plan. You can return multiple values from a function with a [tuple](#), as in:

```
function multiple_return_values(x, y)
    return (x+y, x*y)
end

a, b = multiple_return_values(2, 5)
@show a;
@show b;
```

```
a = 7
b = 10
```

To evaluate the function over vectors of inputs, you can *broadcast* the function by adding a decimal `.` before the function arguments and accessing the resulting values by writing a *comprehension* to loop over the individual outputs in the vector:

```
x = [1, 2, 3, 4, 5]
y = [6, 7, 8, 9, 10]

output = multiple_return_values.(x, y)
a = [out[1] for out in output]
b = [out[2] for out in output]
```

```
@show a;  
@show b;
```

```
a = [7, 9, 11, 13, 15]  
b = [6, 14, 24, 36, 50]
```

```
# function to return the cost and YUK loading as a tuple (cost, loading)  
function compute_results(x1, x2)  
    # compute the cost  
    cost = x1^2/20 + 1.5*x2  
    # compute the loading  
    loading = 0.2*x1 + 0.005*x2^2 + (100 - x1 - x2)  
    return (cost, loading)  
end  
  
@show compute_results(40,40)  
@show compute_results(10, 90)  
@show compute_results(0,0)  
@show compute_results(80,20)
```

```
compute_results(40, 40) = (140.0, 36.0)  
compute_results(10, 90) = (140.0, 42.5)  
compute_results(0, 0) = (0.0, 100.0)  
compute_results(80, 20) = (350.0, 18.0)
```

```
(350.0, 18.0)
```

I created a function that outputs the tuple (cost in \$/day, loading in kg/day) based on the equations created in 2.1. It takes X1 and X2 as arguments and plugs them into the equations. Then I tested the output on 3 different flow combinations to check my answer.

Make sure you comment your code appropriately to make it clear what is going on and why.

Problem 2.3

Use your function to experiment with 1,000 different combinations of wastewater discharge and treatment. You can do this with either a grid search or by sampling from a [Dirichlet distribution](#) (a Dirichlet(1, n) distribution will generate uniformly-weighted n -dimensional vectors whose components add up to 1; see the [Distributions.jl documentation](#) for how to sample from probability distributions in Julia). Plot the results of these experiments. Do any satisfy the YUK effluent standard (plot this as well as a dashed red line). What was the cost

of solutions satisfying the standard? What can you say about the tradeoff between treatment cost and YUK discharge? You don't have to find an "optimal" solution to this problem, but what do you think would be needed to find a better solution?

Problem 2.4

Find the strategies which minimize cost and YUK discharge (these will be different strategies) analytically and find the values of the objective metrics. Plot these values in the plot that you created for Problem 2.3. How do their values compare to the spread of values that you found in that problem? Would you select either of them (explain why or why not)?

References

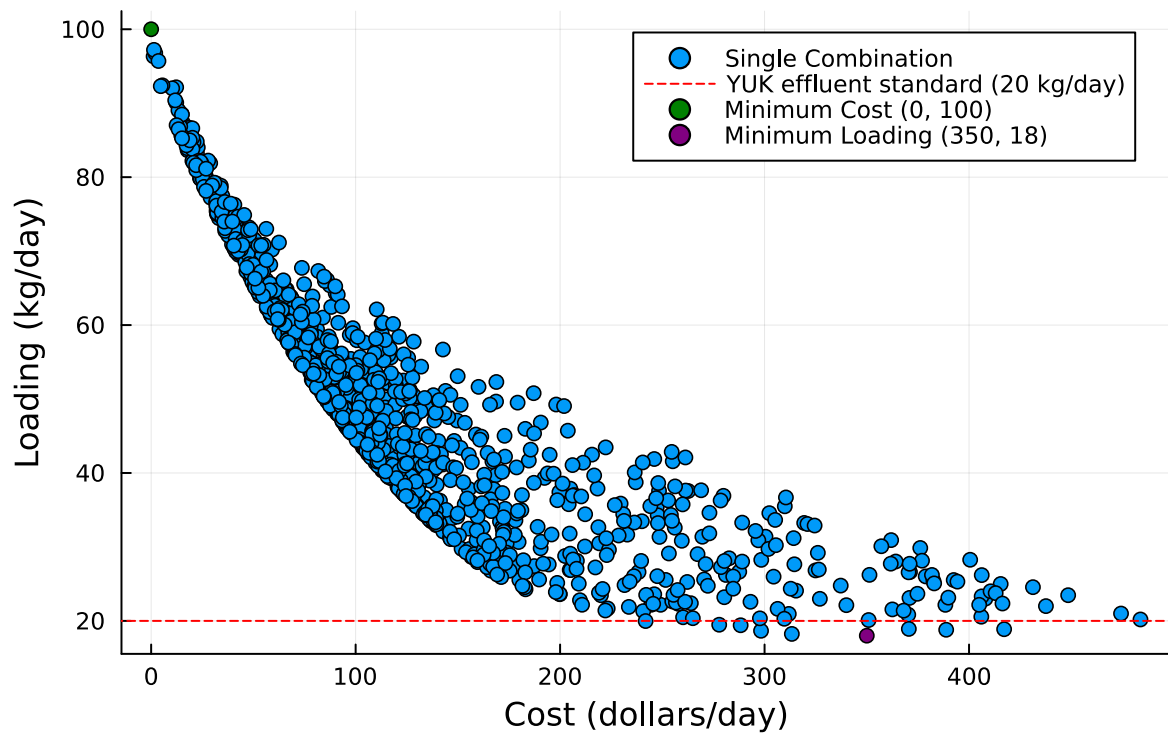
List any external references consulted, including classmates.

```
# add sources and comments
# create dirichlet distribution
dirichlet = Dirichlet(3,1)
# empty arrays to store costs and corresponding loadings
costs = []
loadings = []

# take 1000 points from the distribution and add them to the array
for i in 1:1000
    vect = rand(dirichlet)*100 # multiply by 100 to match total flow of 100 m3/day
    results = compute_results(vect[1], vect[2])
    push!(costs, results[1])
    push!(loadings, results[2])
end

# graph arrays
graph = scatter(costs, loadings, title="Costs vs Loading for 1000 Combinations", xlabel="Cost", ylabel="Loading")
# draw line for YUK effluent standard
hline!(graph, [20], linestyle=:dash, linecolor=:red, label="YUK effluent standard (20 kg/day)")
# draw points for minimized cost and loading
scatter!(graph, [0], [100], color="green", label="Minimum Cost (0, 100)")
scatter!(graph, [350], [18], color="purple", label="Minimum Loading (350, 18)")
```

Costs vs Loading for 1000 Combinations



```
# empty array to store values that meet YUK standard
meet_standard = []

# add to array if loading is less than 20
for i in eachindex(loadings)
    if loadings[i] <= 20
        push!(meet_standard, (costs[i], loadings[i]))
    end
end

# print the costs of values
print("The cost of solutions satisfying the standard are:\n")
for i in eachindex(meet_standard)
    print(string(round(meet_standard[i][1], digits=2)) * " dollars \n")
end
```

The cost of solutions satisfying the standard are:
 461.81 dollars
 389.03 dollars

347.35 dollars
 397.37 dollars
 387.39 dollars
 421.57 dollars
 262.12 dollars
 252.45 dollars
 350.11 dollars
 404.76 dollars
 352.93 dollars

2.3 Answer

Yes, some of the points satisfy the YUK effluent standard, as shown by the points below the red line. The cost of solutions complying with the standard are printed above. They range from around 250 dollars/day to over 400 dollars/day. Generally, cost and loading have a negative correlation. However, one cost can have multiple different loading outcomes and vice versa. Cost and loading have the least correlation when cost approaches around 200 dollars/day. To find a better solution, you could only take flow combinations that already meet the effluent standard. Currently, only a small fraction of the data meets the standard. You would need to find the bounds of the flow combinations that meet the standard, and then take a large sample considering those bounds. This would allow you to get a better idea of the cheapest legal cost and loading combinations.

2.4 Answer

Minimize Cost: To minimize cost, all of the flow should go towards direct disposal. Direct disposal has no costs, while the other two methods have positive costs. A total cost of 0 dollars can be achieved. However this means none of the water will be treated, and the loading will be 100 kg/day.

$$100 \text{ m}^3/\text{day} * 1 \text{ kg}/\text{m}^3 = 100 \text{ kg}/\text{day}$$

$$X_1 = 0, X_2 = 0, \text{Cost} = 0 \text{ dollars}/\text{day}, \text{Loading} = 100 \text{ kg}/\text{day}$$

Minimize Loading: To minimize loading, no flow should go towards direct disposal. Land treatment removes 80% of the YUK, meaning it always treats the water better than direct disposal. Chemical treatment removes $(1 - 0.005X_2)$ of the YUK, meaning it is better at treating the water than direct disposal as long as $X_2 < 200 \text{ m}^3/\text{day}$, which is not possible in this scenario because the total flow is $100 \text{ m}^3/\text{day}$. Therefore $100 = X_1 + X_2$, and the equation for YUK discharge can be rewritten and minimized as shown below.

$$\text{YUK discharge} = 0.2X_1 + 0.005X_2^2 + (100 - X_1 - X_2)$$

$$\text{YUK discharge} = 0.2X_1 + 0.005(100 - X_1)^2$$

$$YUK\ discharge = 0.2X_1 + 0.005(10000 - 200X_1 + X_1^2)$$

$$YUK\ discharge = 0.2X_1 + 50 - X_1 + 0.005X_1^2$$

$$d(YUK\ discharge)/dt = d(0.005X_1^2 - 0.8X_1 + 50)/dt = 0$$

$$0.01X_1 - 0.8 = 0$$

$$X_1 = 80\ m^3/day$$

$$X_2 = 20\ m^3/day$$

$$Treatment\ cost = X_1^2/20 + 1.5X_2\ (dollars/day) = 350\ dollars/day$$

$$YUK\ discharge = 0.2X_1 + 0.005X_2^2 + (100 - X_1 - X_2) = 18\ kg/day$$

Both of the minimized values lie along the shape created by the 1000 points, which is a sign that the values were correctly calculated. The minimized cost value is on the top-leftmost edge of the graph. This makes sense because the cost is exactly zero. The minimized YUK value has the lowest y-value and a somewhat large cost compared to the rest of the points. Each of the minimized points are on nearly opposite sides of the distribution. I wouldn't select either of them because both cost and loading should be considered when deciding on the best option. Minimizing one comes at the expense of the other. The minimized cost case does not meet the effluent standards. While the minimized loading case meets the standard, there are cheaper options that also meet the standard. Based on the distribution, there is a combination of flows that meets the standard and it also much cheaper at around \$250.

Source to create Dirichlet Distribution: <https://juliastats.org/Distributions.jl/v0.14/multivariate.html>

Source to create horizontal line: <https://stackoverflow.com/questions/68171390/hline-vline-with-subplots-in-julia>

Source to create scatterplot: <https://docs.juliaplots.org/dev/tutorial/#Changing-Series-Type:-Scatter-Plots>

Source to round to nearest cent: <https://discourse.julialang.org/t/round-a-float-and-force-a-specific-number-of-decimal-places/87547>